
Simulating a Digital Twin: Spring–Mass Model and Frictional Contact

Department of Mechanical Engineering
Advanced Deep Learning 2026
Joo Yong Sim

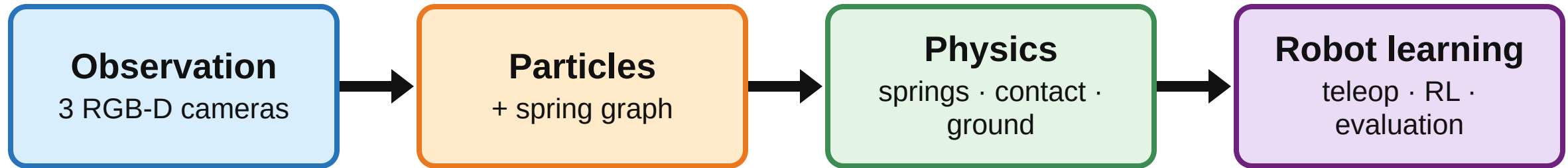


PhysTwin and real2sim-eval read line by line — with measurements from our own experiments on a rope

Lecture Goals: From Video to a Simulator



A digital twin is a physics model that a robot can push, grasp and learn on



Three questions for this lecture

Q1 — *Simulation model* : how does a cloud of particles behave like a rope? (springs, dashpots, drag, integrator)

Q2 — *Contact model* : how do the ground, the rope itself and a robot gripper push back — and how is friction written?

Q3 — *Cost and consequences* : what makes it slow, and what does contact do to repeatability and learning?

Sources : PhysTwin (arXiv 2503.17973) · real2sim-eval (arXiv 2511.04665) · **our projects 13–21 on the released rope twin**

Two Simulators, One Spring Network



Same particles and springs; they differ in how the hand or gripper touches the object

	PhysTwin	real2sim-eval
Purpose	fit physics to a video (system ID)	evaluate robot policies
Hand / gripper	control points welded by springs	finger triangle mesh, frictional contact
Can the object slip or drop?	no — the springs never release	yes — held only by friction
Gradients	differentiable (Warp tape)	forward only, overwritten in place
Self-collision distance	0.02 m (off for real data)	0.005 m (on)
Time step	$\Delta t = 5 \times 10^{-5}$ s, 667 substeps per 30 fps frame (both)	
Rope twin (rope_0001)	2,002 particles · 17,048 object springs · 514 mm long · 31.5 mm across	

real2sim-eval §III-C : “we avoid the common but unrealistic practice of fixing object nodes to the gripper.”

Everything about contact in this lecture follows from that choice.

State and the Spring Graph



Every particle has $\mathbf{x}$, $\mathbf{v}$, $m = 1$; springs join neighbours found at frame 0

Neighbour search (KD-tree)

$$S = \{(i, j) : \|\mathbf{p}_i - \mathbf{p}_j\| < r, j \in \text{kNN}_K(i)\}$$

Defaults $r = 0.02$ m, $K = 30$; the released rope uses $r = 27.4$ mm, $K = 16$.

Rest length — frozen at frame 0

$$L_{0,ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_{t=0}, \quad L_0 > 10^{-4}$$

Rope particles

Count

tracked (from video)

1,495

surface + interior (filled)

288 + 219

median spacing

3.06 mm

PhysTwin [qqtt/engine/trainer_warp.py](#) : spring construction

```
pcd_tree = o3d.geometry.KDTreeFlann(object_pcd)
for i in range(len(points)):
    [k, idx, _] = pcd_tree.search_hybrid_vector_3d(
        points[i], object_radius, object_max_neighbours)
    for j in idx[1:]:
        rest = np.linalg.norm(points[i] - points[j])
        if (i, j) not in seen and rest > 1e-4:
            springs.append([i, j])
            rest_lengths.append(rest)
```

```
masses = np.ones(len(points)) # all particles m = 1
```

```
# repeat 2 (projects 17, 21): 4,004 particles,
# 34,090 springs; scaling tests up to 100,100
```

Spring-Dashpot Force on Each Edge



Hooke's law on strain plus a dashpot along the spring — equal and opposite on the two ends

Unit direction and strain

$$\mathbf{d} = \frac{\mathbf{x}_j - \mathbf{x}_i}{\|\mathbf{x}_j - \mathbf{x}_i\|}, \quad \varepsilon = \frac{\|\mathbf{x}_j - \mathbf{x}_i\|}{L_0} - 1$$

Spring + axial dashpot

$$\mathbf{F}_{ij} = k \varepsilon \mathbf{d} + c [(\mathbf{v}_j - \mathbf{v}_i) \cdot \mathbf{d}] \mathbf{d}$$

$$\mathbf{f}_i += \mathbf{F}_{ij}, \quad \mathbf{f}_j -= \mathbf{F}_{ij}$$

k multiplies a *strain*, so it acts like $E \cdot A$, not N/m. Dashpot $c = 100$.

real2sim-eval sim/physics/spring_mass_warp.py : eval_springs

```
tid = wp.tid() # one thread per spring
if wp.exp(spring_Y[tid]) > spring_Y_min:
    dis = x2 - x1
    dis_len = wp.length(dis)
    d = dis / wp.max(dis_len, 1e-6)
    spring_force = (
        wp.clamp(wp.exp(spring_Y[tid]),
                  low=spring_Y_min, high=spring_Y_max)
        * (dis_len / rest - 1.0) * d)
    v_rel = wp.dot(v2 - v1, d)
    dashpot_forces = dashpot_damping * v_rel * d
    overall_force = spring_force + dashpot_forces
    wp.atomic_add(f, idx1, overall_force)
    wp.atomic_sub(f, idx2, overall_force)
```

Note the atomics : many springs add into the same particle in parallel — the summation order is not fixed (we return to this at the end).

Stiffness Is Stored as a Logarithm



One stiffness per spring, optimised in log space and clamped to a range

Parameterisation

$$k_s = \text{clip}(e^{Y_s}, Y_{\min}, Y_{\max})$$

$$Y_{\min} = 0, \quad Y_{\max} = 10^5$$

Why log : $k > 0$ for free, and a gradient step changes k by a *ratio*, not an amount. Initial $k = 3 \times 10^4$.

Trap : `case.spring_Y` is linear, `sim.wp_spring_Y` is its log — exponentiate the wrong one and every spring sits at the 10^5 clamp.

float32 : $\|\mathbf{x}\|/L_0 - 1$ cancels near 1 — **2.3 %** force error vs float64.

Fitted stiffness, rope_0001

Quantile	k
min	11.5
p10	1,821
median	43,785
p90	100,797
max	102,244

p10–p90 spread **55×** here vs 1.05× in PhysTwin's `single_push_rope` — many springs ride the upper clamp.

Gravity, Drag and Semi-Implicit Euler



Velocity first from forces, then position from the new (contact-corrected) velocity

1. Velocity from force, then drag

$$\mathbf{v}^+ = \left(\mathbf{v} + \frac{\mathbf{f} + m\mathbf{g}}{m} \Delta t \right) e^{-c_{\text{drag}} \Delta t}$$

$c_{\text{drag}} = 3 \text{ s}^{-1}$: exponential decay, never overshoots zero.

2. Contacts modify $\mathbf{v}^+$

3. Position (symplectic Euler)

$$\mathbf{x}^+ = \mathbf{x} + \mathbf{v}^+ \Delta t$$

$$n_{\text{sub}} = \text{round}\left(\frac{1/30}{5 \times 10^{-5}}\right) = 667$$

spring_mass_warp.py : update_vel_from_force

```
tid = wp.tid()                                # one thread per particle
v0 = v[tid]
f0 = f[tid]
m0 = masses[tid]

drag_damping_factor = wp.exp(-dt * drag_damping)
all_force = f0 + m0 * wp.vec3(0.0, 0.0, -9.8) \
            * reverse_factor    # PhysTwin: z down

a = all_force / m0
v1 = v0 + a * dt
v2 = v1 * drag_damping_factor
v_new[tid] = v2
```

Upstream quirk : with a mesh present, mesh_collision already writes $\mathbf{x} + \mathbf{v}\Delta t$ and the ground kernel advances again — **free fall measured at 2.002× analytic**. The released parameters were fitted with it.



Why 667 Substeps: Explicit Stability

An explicit integrator must resolve the fastest spring — the stiffest, shortest one

One spring, one mass: $\omega = \sqrt{k_{\text{eff}}/m}$

$$k_{\text{eff}} = \frac{k}{L_0} \quad (\text{force per metre})$$

Symplectic Euler is stable for $\omega\Delta t < 2$

$$\Delta t < 2\sqrt{\frac{m L_0}{k}} = 126.7 \mu\text{s}$$

with $k_{\text{max}} = 10^5$, shortest $L_0 = 0.40$ mm. The dashpot limit $2m/c = 20$ ms is far looser.

Released $\Delta t = 50 \mu\text{s}$ sits **within 1.7×** of the spring limit.

Measured on the rope (projects 13, 21)

Substeps	Δt (μs)	Result
667	50	reference
500	67	stable, 13.3 mm rms · 1.21×
444	75	stable, 8.3 mm drift · 1.49×
400	83	stable, 18.2 mm
334	100	diverges (NaN)

Rerun spread of the reference itself is 16.6 mm, so 500 substeps is “free”.

PhysTwin Control: Points Welded by Springs



Tracked hand points are kinematic; springs tie them to nearby object particles

Hand points

Grounded-SAM2 hand masks + CoTracker →
lifted to 3D → farthest-point sampled.

Attachment (frame 0, never changes)

$\|\mathbf{c}_a - \mathbf{p}_j\| < 0.04 \text{ m}, \leq 50 \text{ per point}$

Inside a frame: linear interpolation

$$\mathbf{c}(s) = \mathbf{c}_{\text{prev}} + (\mathbf{c}_{\text{target}} - \mathbf{c}_{\text{prev}}) \frac{s + 1}{n_{\text{sub}}}$$

What this boundary condition means

- A **soft Dirichlet** condition: prescribed motion through a compliant spring
- Pulls as hard as it pushes
- Never releases, never slips
- Sparse: in single_push_rope only **2 of 30** hand points are within 40 mm

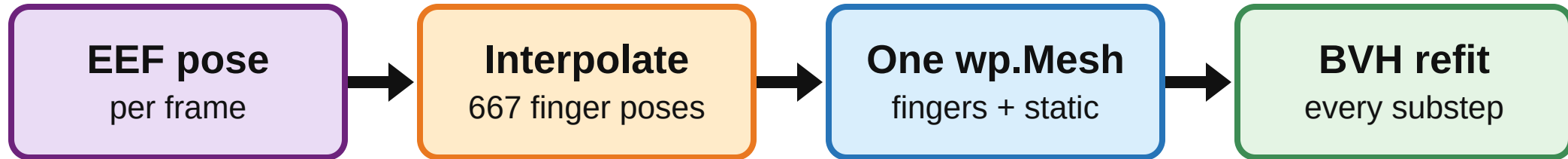
Code note: wp_control_v is never written, so the dashpot on control springs sees a control velocity of 0.

Good for fitting a video, wrong for robot grasping: a policy could “lift” the rope without ever squeezing it.

real2sim-eval: Gripper as a Moving Mesh



Finger meshes and static scenery are one triangle mesh with a BVH, refit every substep



Rigid velocity of a finger point

$$\mathbf{V}(\mathbf{x}) = \mathbf{V}_{\text{finger}} + \boldsymbol{\omega} \times (\mathbf{x} - \mathbf{c}_{\text{eef}})$$

xArm fingers: 88 triangles, 48 vertices.

mesh_map[face] = 0 / 1 for the fingers, < 0 for static meshes.

Grasp is a force latch, not a weld

- Read normal “force” on fixed finger faces
- **Latch** when both fingers exceed a threshold (3×10^4 ; 4,386 after our recalibration)
- **Release** when both drop below a low threshold

One Contact Law: Restitution and Coulomb



Ground, self- and mesh contact all edit the velocity — no penalty springs

Split relative velocity along the normal $\mathbf{n}$

$$\mathbf{v}_n = (\mathbf{v} \cdot \mathbf{n}) \mathbf{n}, \quad \mathbf{v}_t = \mathbf{v} - \mathbf{v}_n$$

Restitution $e \in [0, 1]$

$$\mathbf{v}_n' = -e \mathbf{v}_n \Rightarrow |\Delta \mathbf{v}_n| = (1 + e) |\mathbf{v}_n|$$

Coulomb: $|\Delta \mathbf{v}_t| \leq \mu |\Delta \mathbf{v}_n|$

$$\mathbf{v}_t' = a \mathbf{v}_t, \quad a = \max \left(0, 1 - \mu (1 + e) \frac{|\mathbf{v}_n|}{|\mathbf{v}_t|} \right)$$

Reading the scale factor a

Case	Meaning
$a = 0$	stick : friction impulse can stop all sliding
$0 < a < 1$	slip : slide slowed by exactly $\mu \cdot$ normal impulse
$a \rightarrow 1$	grazing, $ \mathbf{v}_n \ll \mathbf{v}_t $

Clamps in the kernels

$$e \in [0, 1], \quad \mu \in [0, 2]$$

Same idea as the PushT impulse model — but applied per particle, per substep, with unit mass and no rotation.

Ground Contact with Time of Impact



If the next step would cross $z = 0$ while moving down, apply the law and split the step

Trigger

$$z + v_z \Delta t < 0 \text{ and } v_z < -10^{-4}$$

Time of impact, then two half-moves

$$t_c = -\frac{z}{v_z}, \quad \mathbf{x}^+ = \mathbf{x} + \mathbf{v} t_c + \mathbf{v}'(\Delta t - t_c)$$

Resting particle, $e = 0$

$$|\mathbf{v}'_t| = |\mathbf{v}_t| - \mu g \Delta t$$

Gravity re-supplies $g\Delta t$ of normal speed each substep $\rightarrow$ constant deceleration μg — textbook sliding friction.

spring_mass_warp.py : integrate_ground_collision

```
normal = wp.vec3(0.0, 0.0, 1.0) * reverse_factor
next_x_z = (x_z + v_z * dt) * reverse_factor
if next_x_z < 0.0 and v_z * reverse_factor < -1e-4:
    v_normal = wp.dot(v0, normal) * normal
    v_tao = v0 - v_normal
    v_normal_new = -clamp_collide_elas * v_normal
    a = wp.max(0.0, 1.0 - clamp_collide_fric
               * (1.0 + clamp_collide_elas)
               * v_normal_length / v_tao_length)
    v1 = v_normal_new + a * v_tao
    toi = -(x_z - ground_height) / v_z
else:
    v1 = v0;   toi = 0.0
x_new[tid] = x0 + v0 * toi + v1 * (dt - toi)
```

Self-Collision: Hash Grid, Then Impulses



Particles of different parts that approach closer than 5 mm exchange an impulse

Broad phase (once per frame)

wp.HashGrid, cell $5d_c$; keep pairs with different masks and $\|\mathbf{x}_j - \mathbf{x}_i\| < d_c$ (≤ 500 per particle). Pairs already close at frame 0 are excluded.

Narrow phase: approaching pairs only

$$(\mathbf{x}_j - \mathbf{x}_i) \cdot (\mathbf{v}_j - \mathbf{v}_i) < -10^{-4}$$

$$\mathbf{J} = \frac{-(1 + e_s) \mathbf{v}_n^{\text{rel}} + (a - 1) \mathbf{v}_t^{\text{rel}}}{1/m_i + 1/m_j}$$

$$\mathbf{v}_i' = \mathbf{v}_i - \frac{1}{m_i} \bar{\mathbf{J}} \quad (\text{mean over contacts})$$

Parameters

	Value
d_c	0.005 m (PhysTwin 0.02)
e_s, μ_s	clamped to [0,1], [0,2]
averaging	Jacobi-style mean

What we found

In our project-13 bridge, `update_collision_graph()` was never called: **the kernel ran 667×/frame and looped zero times**. Upstream's driver does call it.

Mesh Contact: Query, Respond, Project



Every particle asks the BVH: “will I be inside a finger or the table next step?”

1. Predict and query (signed distance)

$$\mathbf{x} = \mathbf{x} + \mathbf{v}\Delta t, \quad \text{err} = \text{sign} \cdot \|\mathbf{x} - \mathbf{p}\| - \delta$$

inside/outside by winding number; margin $\delta = 5$ mm (finger), 1 mm (static).

2. If $\text{err} < 0$: law in the finger's frame

$$\mathbf{v}^{\text{rel}} = \mathbf{v} - \mathbf{V}(\mathbf{x}), \quad \mathbf{v}' = C_{e,\mu}(\mathbf{v}^{\text{rel}}) + \mathbf{V}(\mathbf{x})$$

Fingers: $e = 0$, $\mu = 1.0$. Static: ground values.

3. Project out, read force

$$\mathbf{x} \leftarrow \mathbf{x} - \mathbf{n} \text{err}, \quad F_{\text{face}} += \Delta v_n / \Delta t$$

spring_mass_warp.py : mesh_collision (condensed)

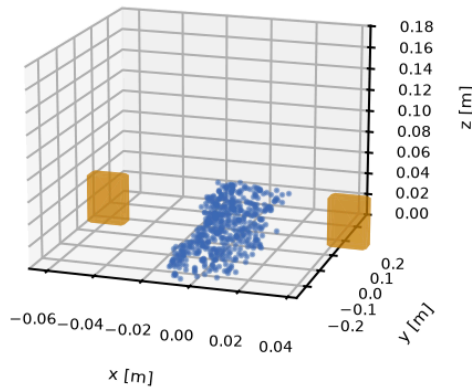
```
next_x = x0 + v0 * dt
query = wp.mesh_query_point_sign_winding_number(
    mesh, next_x, max_dist=0.02,
    accuracy=3.0, threshold=0.6)
p = wp.mesh_eval_position(mesh, query.face, ...)
err = wp.length(next_x - p) * query.sign - margin
if err < 0.0:
    normal = wp.normalize(next_x - p) * query.sign
    real_dynamic_velocity = dynamic_velocity[k] \
        + wp.cross(dynamic_omega[0], x0 - center)
    v0 = v0 - real_dynamic_velocity
    # ... restitution + Coulomb scale a ...
    next_v = v_normal_new + a * v_tao \
        + real_dynamic_velocity
    # finger: re-predict, query again, push out
    next_x = next_x - normal * err
    wp.atomic_add(collision_forces, face,
        delta_v_normal / dt)
```

Frictional Grasp: Lifted Without a Weld

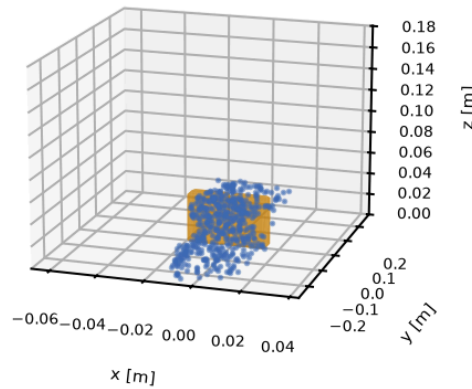


Approach, close, lift: the rope rises only through the friction its own squeeze creates

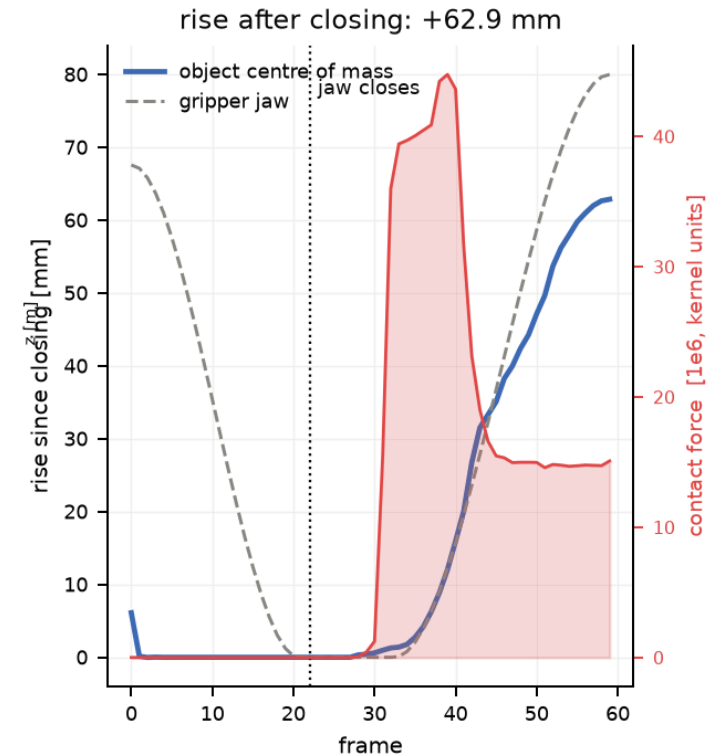
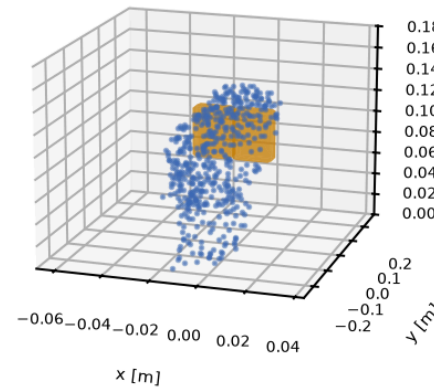
rope · approach (frame 16)



rope · jaw closed (frame 32)



rope · lifted (frame 59)



Project 13: orange = finger meshes, blue = rope particles. Right: centre-of-mass rise (+62.9 mm), jaw opening (dashed), contact force (red).

Read the plot : the force spikes when the jaw closes, then settles at the value that holds the rope's weight — **grip force $\times \mu \geq \text{weight}$** , or it slips.

One Substep, Kernel by Kernel



Six GPU kernels and one BVH refit — repeated 667 times for each 33 ms video frame



Per frame (30 fps)

- Once: hash grid, 667 interpolated finger poses
- $667 \times (6 \text{ kernels} + 1 \text{ refit}) \approx \mathbf{4,000 \text{ launches}}$
- All kernels: one thread per spring or per particle

Colour key

blue spring–mass–damper · **orange** object contact · **purple** gripper geometry · **green** ground + integration

PhysTwin: same order, no mesh stages, plus *set_control_points* first.

Where the Time Goes (GPU, 4,004 Particles)



Doubling the particles costs nothing — the simulator is launch-latency bound

Part (667 substeps)	ms/frame	Share
BVH refit (88 triangles)	29.5	28.0 %
eval_springs (34,090)	22.0	20.8 %
mesh_collision	14.6	13.8 %
object_collision (inert)	10.7	10.2 %
ground + integrate	9.8	9.3 %
update_vel	8.5	8.1 %
set_mesh_points	6.5	6.1 %
clear_forces	4.0	3.7 %

Fit over substep counts

$$t_{\text{frame}} \approx 8.7 \text{ ms} + 0.121 \text{ ms} \times n_{\text{sub}}$$

2,002 vs 4,004 particles: **89.3 vs 88.8 ms**.

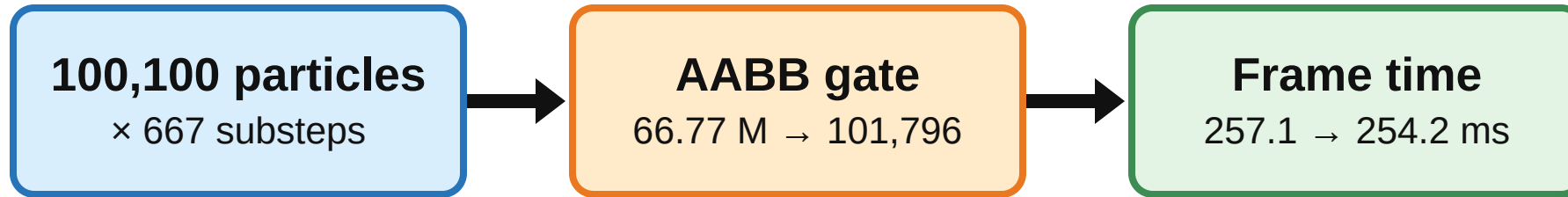
Grouped : gripper contact path ≈ 46 %, springs ≈ 33 %, ground ≈ 9 %.

Refitting a **48-vertex** mesh costs more than 34k springs — it is pure launch overhead.

Lesson: A Broad Phase Did Not Help



Filtering particles with the gripper's box cut queries 656× — and the frame by 1.1 %



Cost per particle query (GPU)

Case	Time
BVH miss	0.007 ns
AABB test	0.018 ns
BVH hit	20 ns (2,933× a miss)

Why

The BVH's first box test *already is* a broad phase. Only 3.8 % of (particle, substep) pairs are real hits, and those are what cost time.

Profile before you optimise — the textbook fix attacked a cost that did not exist.



What Actually Made It Faster

Remove launches, atomics and refits: 1.7× at 4k particles, 1.9× at 100k

Change	4,004	100,100
cache clip(e^Y)	+0.2 ms	−5.0 ms
per-particle spring gather, no atomics	+1.6 ms	−57.6 ms
fuse mesh contact + ground	−2.0 ms	−5.1 ms
static finger BVH in local frame	−11.8 ms	−11.6 ms
CUDA graph replay	−12.0 ms	−14.8 ms
total (with 500 substeps)	81.6 → 48.4	264.3 → 137.8

Two key tricks

Gather : each particle sums its own springs —

$$\mathbf{f}_i = \sum_j [k \varepsilon_{ij} + c (\mathbf{v}_j - \mathbf{v}_i) \cdot \mathbf{u}] \mathbf{u}$$

Local BVH : move the particle, not the mesh —

$$\mathbf{x}_\ell = R^\top (\mathbf{x}_w - \mathbf{t})$$

Result : 3,335 launches → 1 per frame, 2 kernels per substep, every rms inside the 22.9 mm upstream self-spread.

CPU vs GPU: The Ranking Inverts



CPU: a spring solver with some contact — GPU: a contact solver with some springs

Role (4,004 particles)	CPU ms	GPU ms	CPU/GPU
spring–mass–damper	201.8 (60.5 %)	14.9 (15.4 %)	13.5×
gripper contact	103.1 (30.9 %)	66.7 (68.9 %)	1.5×
integration	16.6 (5.0 %)	7.3 (7.5 %)	2.3×
ground	12.2 (3.7 %)	7.9 (8.2 %)	1.5×
BVH refit alone	0.5	19.1	0.03×

Warp's CPU backend is a serial loop, so it measures *work*; the GPU measures latency.

Same change, two verdicts

Change	CPU	GPU
cache exp	−16.2 %	−0.6 %
local BVH	+5.9 %	−16.5 %
broad phase	≈ 0	≈ 0

GPU speed-up grows with size: 3.2× at 2k → **24×** at 100k particles.

Reduced Models: Fewer Degrees of Freedom



Project the physics onto a few coarse modes; treat contact as a constraint, not a force

18 — latent operator, not learned

$$K_r = P^\top K P, \quad M_r \ddot{\mathbf{z}} = P^\top \mathbf{f} - K_r \mathbf{z} - C_r \dot{\mathbf{z}}$$

6,006 DOF $\rightarrow$ 96. K_r is the exact Schur complement (8e-14). A whole frame = one matvec with Φ^{667} .

5.10× end-to-end at 2.32 mm (0.23 %) error.

19 — reduced contact constraint

$$J_r = J P, \quad p_n \geq 0, \quad \|\mathbf{p}_t\| \leq \mu p_n$$

Coulomb *disk*, solved by PGS; 4 implicit microsteps replace 667 substeps.

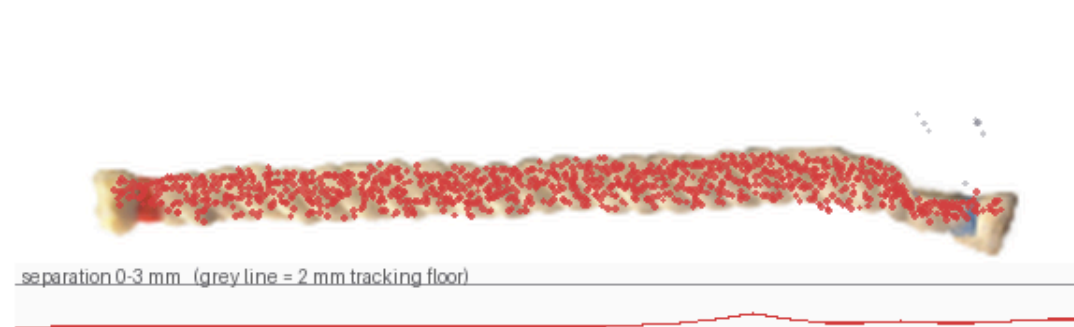
Scene	Speed-up
resting / sliding	3.78× / 2.55×
3 cm drop	2.89×
gripper approach	1.18×, 90 mm worst — failure

Contact Makes the Simulator Non-Repeatable

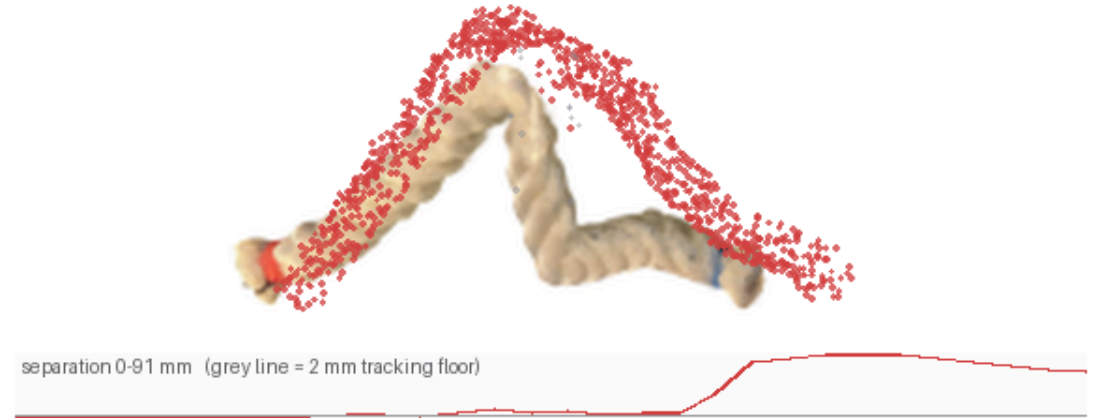


Identical inputs, different rope: atomic float sums reorder, and contact amplifies it

quasi_static_press two identical runs overlaid
frame 30/30 separation 0.38 mm never separates



lift_drop two identical runs overlaid
frame 30/30 separation 66.21 mm apart from frame 9



Project 15: two identical runs overlaid (red = second run). Left: quasi-static press, 0.38 mm. Right: lift and drop, 66 mm.

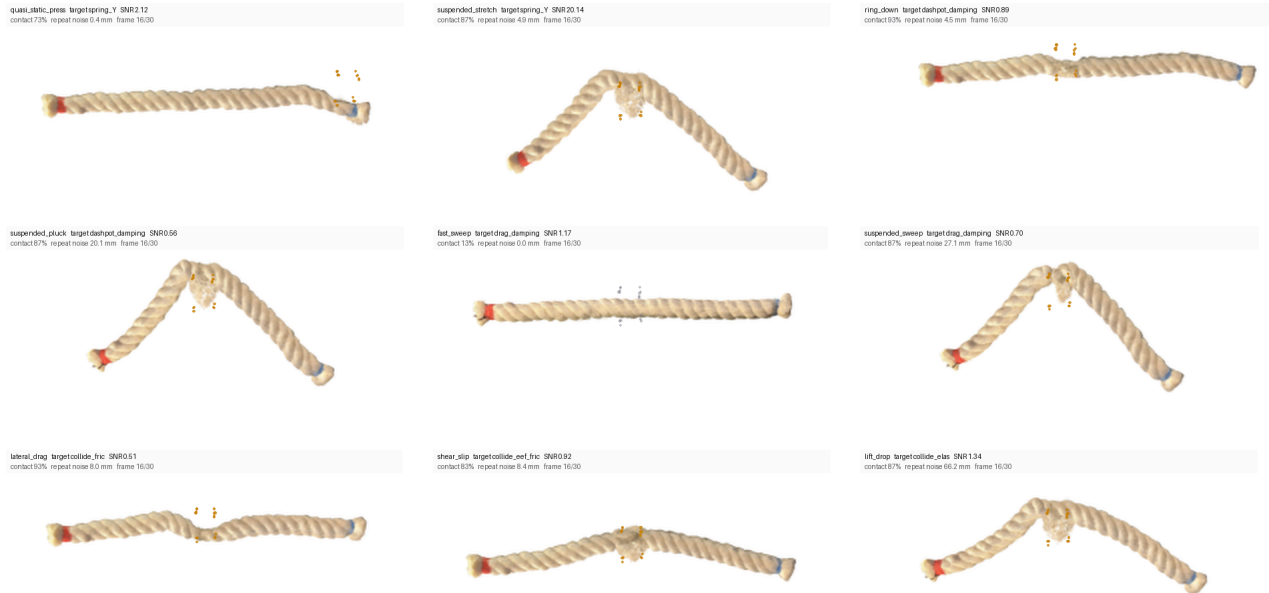
Why : $(a + b) + c = a + (b + c)$ in float32. A tiny difference decides whether a particle is inside the 5 mm margin — **a contact appears or not**.

8 replays of one nominal (RL, 17) : 27.2, 30.5, 30.3, 27.7, 27.9, **50.6**, 30.2, 29.3 mm. The per-particle gather is bit-reproducible.

Contact Churn Limits What We Can Identify



Probes designed to excite one parameter also create the most noise



Project 15: nine textbook probes, each with contact %, repeat noise, SNR

Probe	Repeat noise
quasi-static press	1.2 mm
ring-down	3.7 mm
lateral drag	7.1 mm
lift-drop	44–99 mm

Noise tracks **churn** (contacts forming and breaking), not contact time.

Fisher design predicted 1.19×; actual fits: **random wins 5/5** ($p \approx 0.004$).



A rope twin is springs + one velocity-level contact law, run 667 times per frame

- **Simulation model** : log-stiffness springs with axial dashpots, exponential drag, symplectic Euler at 50 μs
 - Δt is set by the stiffest, shortest spring — 1.7× below the limit
- **Boundary condition** : PhysTwin welds the hand with springs; real2sim-eval uses a moving mesh with friction
 - only the second can slip, drop, and be a fair test of a grasp
- **Contact model** : $\mathbf{v}'_n = -e\mathbf{v}_n$, $\mathbf{v}'_t = a\mathbf{v}_t$ for ground, self and mesh
 - Coulomb friction as a velocity scale factor, plus position projection
- **Cost** : on a GPU, launches and refits dominate — not arithmetic, not the broad phase
- **Consequence** : **contact turns float round-off into mm-scale divergence** — budget for it in RL and system ID



Exercises with the simulator

- **Stability** : derive $\Delta t_{\max}$ for a chain of 3 masses — when is it tighter than one spring?
- **Friction** : show that $a = 0$ exactly when the needed friction impulse is $\leq \mu(1 + e) |\mathbf{v}_n|$
- **Grasp** : with $\mu = 1$, what squeeze force holds a 50 g rope? Compare with the latch threshold
- **Weld vs friction** : replay one trajectory in both models — where do they disagree?
- **Determinism** : run the same frame 10 times with atomics, then with the gather kernel
- **Profiling** : predict the CPU and GPU ranking of each kernel before you measure it